QUICK REFERENCE

Design Patterns Cheat Sheet

All 23 patterns: the idea, the real problem each one solves, and the words in a question that hint at it.

CREATIONAL (5) How objects get made

PATTERN	THE IDEA	REAL PROBLEM IT SOLVES	INTERVIEW SIGNAL
Singleton	Only one instance, shared everywhere.	Two copies of a config or connection pool cause conflicts and waste.	*exactly one, shared across the app, global access*
Factory	One place decides which object to create.	`new` calls scattered everywhere, so each new type means edits in many files.	*create different types from input or config*
Abstract Factory	Create whole families of matching objects.	Mixing a dark button with a light dialog, or one vendor's pieces with another's.	*themes, platforms, vendors, must match*
Builder	Build a complex object step by step.	Constructors with 8 optional arguments that nobody can read.	*many optional parameters, step by step*
Prototype	Copy an existing object, then tweak it.	Building from scratch is slow, but a copy with small changes is cheap.	*clone, template object, expensive to create*

STRUCTURAL (7) How objects fit together

PATTERN	THE IDEA	REAL PROBLEM IT SOLVES	INTERVIEW SIGNAL
Adapter	Sit between two things that do not fit.	A third party or legacy API has the wrong shape for your code.	*integrate, legacy, incompatible interface*
Decorator	Add abilities in layers, original untouched.	A class for every combination of extras (subclass explosion).	*add behavior dynamically, combine features*
Facade	One simple front for a complicated system.	Callers must know six services and their order to do one thing.	*simplify, single entry point, hide complexity*
Proxy	A stand-in that controls access.	Need access checks, caching or lazy loading without changing the real object.	*control access, lazy load, cache, log calls*
Composite	Treat one item and a group the same way.	Code that special-cases "one item" versus "a group" everywhere.	*tree, nested, same action on one or many*
Bridge	Split "what it does" from "how it does it".	Classes multiply when two things vary, like shape times renderer.	*two dimensions of change, switch at runtime*
Flyweight	Share the heavy, identical parts.	Thousands of near identical objects use up memory.	*huge number of objects, memory, shared data*

QUICK REFERENCE

Behavioral patterns

How objects talk to each other and share the work.

BEHAVIORAL (11) How objects talk and share work

PATTERN	THE IDEA	REAL PROBLEM IT SOLVES	INTERVIEW SIGNAL
Observer	Subscribe once, get told on change.	One change must update several parts, and the source should not know them.	*notify, subscribe, events, when X then Y and Z*
Strategy	Swap the "how" without touching the rest.	A long if/else that picks an algorithm and grows with every option.	*multiple ways to, swap at runtime, pricing rules*
Command	Turn a request into an object.	Need undo, retries, queues or a log of actions.	*undo, redo, queue, schedule, macro*
State	Behavior changes with the current state.	The same status checks repeated across many methods.	*workflow, lifecycle, depends on status*
Chain of Responsibility	Pass it along until someone handles it.	The sender should not know who handles it, and checks run in order.	*middleware, approval levels, pipeline of checks*
Template Method	Fixed recipe, subclasses fill in steps.	Two classes follow the same steps, so the code gets copy pasted.	*same algorithm, different steps, hooks*
Iterator	Walk a collection one item at a time.	Loops that break when you change how the data is stored.	*traverse without exposing internals*
Mediator	One go-between for everyone.	Components talk to each other directly and break when one changes.	*chat room, form fields reacting to each other*
Memento	Save a snapshot to go back to.	Restore an earlier state without exposing the object's insides.	*checkpoint, snapshot, rollback*
Visitor	Add new jobs without changing the objects.	Every new operation means editing every class.	*new operations on a stable set of types*
Interpreter	Each rule of a small language is a class.	Evaluating small rules or expressions that arrive as text.	*mini language, rules engine, expression*

IN AN INTERVIEW

Describe the problem first, then name the pattern, then say the trade-off. Patterns add structure, so mention when a plain function or an if statement is enough.

PRACTICE

Name the pattern

23 problems, one for each pattern, with no pattern names. Try them first, then check the answer key on the next page.

1. Charge customers through Stripe today and PayPal next month, picked per user at runtime.
2. When an order is placed, send an email, update stock and log analytics. A new reaction should not touch the order code.
3. A text editor needs undo and redo for every action.
4. An old SMS library has methods that do not match your NotificationService interface.
5. Every request must pass auth, then rate limiting, then validation, and any step can stop it.
6. The whole app needs one shared logger that is never created twice.
7. Customers add any mix of milk, caramel and extra shot to a drink, and prices add up. No class per combination.
8. Checkout calls inventory, payment, shipping and email in a fixed order. The frontend wants one call: placeOrder().
9. A page should load a heavy video only when the user scrolls to it.
10. A game draws 100,000 trees that differ only by position, and memory is running out.
11. An order moves from Draft to Review to Approved to Published, and the allowed actions change at each step.
12. A report exports to PDF, CSV and Excel. All do: fetch data, format it, save it. Only formatting differs.
13. A menu has items and submenus. Getting the total price must work the same for one item or a whole submenu.
14. Build an HTTP request with URL, headers, timeout, retries, auth and body. Most are optional.
15. In a form, changing one field enables or hides others, but fields should not know about each other.
16. Create buttons and dialogs for either Windows or Mac, and they must always match each other.
17. Draw circles and squares on canvas or SVG without writing CircleCanvas, CircleSVG, SquareCanvas...
18. Calculate area, export JSON and validate every node of a syntax tree without editing the node classes each time.
19. Users type price > 100 AND category = "books" and your app must evaluate it.
20. Loop over a custom tree with a plain for...of, without exposing how it is stored.
21. Save a checkpoint before a boss fight and restore it if the player dies.
22. Spawn many enemies that start as copies of one fully configured template enemy.
23. Create email, SMS or push notification objects from a type string in config, without `new` scattered everywhere.

PRACTICE

Answer key

1	**Strategy**	Swappable payment algorithms behind one interface.
2	**Observer**	The order publishes an event and listeners subscribe.
3	**Command**	Each action is an object that can run and undo itself. Memento also works.
4	**Adapter**	Wrap it so it looks like the interface you expect.
5	**Chain of Responsibility**	Each check handles the request or passes it on, like middleware.
6	**Singleton**	One instance, one global access point.
7	**Decorator**	Wrap the drink in layers, one per extra.
8	**Facade**	One simple method hides the subsystem calls.
9	**Proxy**	A stand-in loads the real thing on first use (lazy loading).
10	**Flyweight**	Share the model and texture, store only the position.
11	**State**	Each status is its own object with its own behavior.
12	**Template Method**	The base class fixes the steps, subclasses fill in the format.
13	**Composite**	Treat leaves and groups through the same interface.
14	**Builder**	Set only what you need, step by step, then build.
15	**Mediator**	Fields talk to one coordinator, not to each other.
16	**Abstract Factory**	One factory per platform makes the whole matching family.
17	**Bridge**	Split shapes from renderers and combine them freely.
18	**Visitor**	Each new job is a new visitor, nodes stay untouched.
19	**Interpreter**	Each rule is a class that can compute itself.
20	**Iterator**	The iterator tracks the position and hands out the next item.
21	**Memento**	Snapshot the state outside the object and restore it later.
22	**Prototype**	Clone the template instead of building each from scratch.
23	**Factory**	One factory function decides which object to return.

Code, diagrams and runnable examples
https://github.com/misbahafzal/design-patterns-made-easy
Read the full series on Medium
Part 1: Software Design Patterns Made Easy